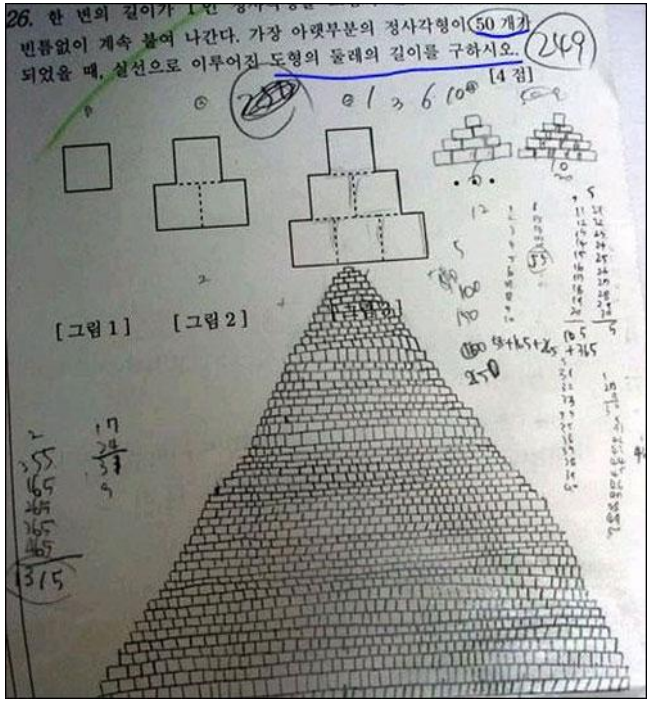
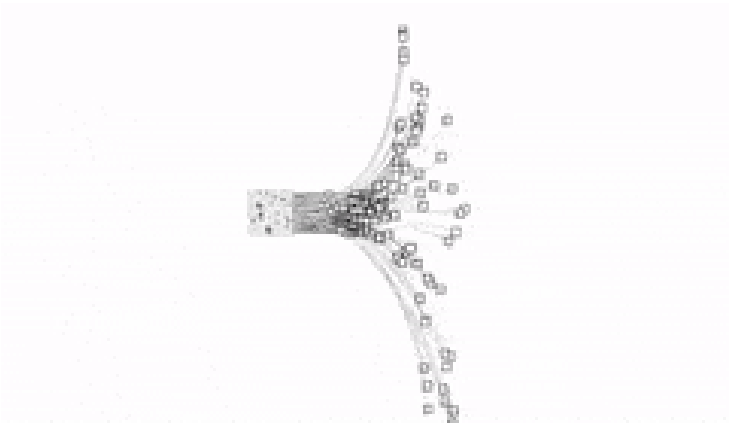
Divide and Conquer

이원형 교수
Dr. Lee, WonHyong
whlee@handong.edu

DESIGN PARADIGMS

- Incremental approach
- Divide and Conquer
- Dynamic Programming
- Greedy
- Branch and Bound
- Brute force ?





$2.08 \times 10^{170} \gg 12 \times 10^{78}$

RECURRENCE

RECURRENCE EQUATION

- In many cases we can use a ***recurrence equation*** to describe the running time of an algorithm.
- In this chapter we will study how to solve recurrence equation.

https://levelup.gitconnected.com/how-to-think-recursively-solving-recursion-problems-in-4-steps-95a6d07aa866

REMARKS

- Difference between recurrence and recursion?
  - ✓ **Recurrence** is a mathematical concept that refers to a sequence of numbers or functions that are defined recursively.
    - Ex) Fibonacci recurrence relation

$$F(n) = F(n-1) + F(n-2)$$

where $F(0) = 0$ and $F(1) = 1$ are the initial values.

  - ✓ **Recursion**, on the other hand, is a programming technique where a function calls itself in order to solve a problem.
    - Ex) Fibonacci recursion approach

```
int fibonacci(int n) {
    if (n <= 1) {
        return n;
    } else {
        return fibonacci(n-1) + fibonacci(n-2);
    }
}
```

Recurrence Equation

- **The expression:**

$$T(\boldsymbol{n}) = \begin{cases} c & \text{if } n = 1 \\ 2T(\boldsymbol{n}/2) + c\boldsymbol{n} & \text{if } n > 1 \end{cases}$$

  - ✓ is a recurrence.

- **Technicalities**
  - ✓ Floors and ceilings
  - ✓ Exact vs. asymptotic functions
  - ✓ Boundary conditions

❖ Recurrence Examples

- ✓ $S(n) = \begin{cases} 0 & \text{if } n = 0 \\ c + S(n-1) & \text{if } n > 0 \end{cases}$

- ✓ $S(n) = \begin{cases} 0 & \text{if } n = 0 \\ n + S(n-1) & \text{if } n > 0 \end{cases}$

- ✓ $T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$

- ✓ $T(n) = \begin{cases} c & \text{if } n = 1 \\ aT(n/b) + cn & \text{if } n > 1 \end{cases}$

Solving Recurrence Equations

- Iteration method
- Substitution method
- Recursion-tree method
- Master method

Iteration Method

Iteration Method

- **One option is the "Iteration method"**
  - ✓ Expand the recurrence
  - ✓ Work some algebra to express as a summation
  - ✓ Evaluate the summation

- **We will see a few examples.**

EXAMPLE

- $s(n)$

  $= c + s(n-1)$

  $= c + c + s(n-2)$

  $= 2c + s(n-2)$

  $= 2c + c + s(n-3)$

  $= 3c + s(n-3)$

  …

  $= kc + s(n-k)$ and $k < n$

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

EXAMPLE

- **So far for $k < n$, we have**
  - ✓ $s(n) = ck + s(n-k)$
- **What if $k = n$?**
  - ✓ $s(n) = cn + s(0) = cn$
- **Thus, in general**
  - ✓ $s(n) = cn$
  - ✓ *"Are you sure about this conclusion?"*

$$s(n) = \begin{cases} 0 & n = 0 \\ c + s(n-1) & n > 0 \end{cases}$$

Proof By Induction

- **Base case**: $n = 0$ **or** $n = 1$
  - ✓ Show formula is true when $n = 0$ or 1

- **Inductive hypothesis**:
  - ✓ For n greater than the base case,
    assume that the formula holds true for all $0 \leq k < n$
  - ✓ By inductive hypothesis, the formula is true when $k = n - 1$

- **Proof of goal statement**:
  - ✓ Show that formula is then true for n

EXERCISE

- **Prove** $1^2 + 2^2 + 3^2 + \ldots + n^2 = n(n+1)(2n+1) / 6$
  - ✓ Base case:
    - When $n = 1$, then $1 = 1(1+1)(2+1) / 6$
  - ✓ Inductive hypothesis:
    - For n greater than 0, assume that $1^2 + 2^2 + 3^2 + \cdots + k^2 = k(k+1)(2k+1)/6$ holds true all $0 \le k < n$.
    - By the hypothesis, the formula is also true when $k = n - 1$ (because $0 \le n - 1 < n$).

      $1^2 + 2^2 + 3^2 + \ldots + (n-1)^2 = (n-1)n(2n-1) / 6$
  - ✓ Proof of goal statement:

    $1^2 + 2^2 + \ldots + (n-1)^2 + n^2 = (1^2 + 2^2 + \ldots + (n-1)^2) + n^2$

    $= (n-1)n(2n-1) / 6 + n^2 = n(n+1)(2n+1) / 6$

EXERCISE

- **Prove** $1 + 2 + 3 + \ldots + n = n(n+1) / 2$
  - ✓ Base case:
  - ✓ Inductive hypothesis:
  - ✓ Proof of goal statement:

Substitution Method

SUBSTITUTION METHOD

- Guess the form of the solution.
- Prove the guess is correct by mathematical induction or mathematical tautology(동의어).

❖ **Example)** MergeSort Time Complexity

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

✓ Guess

$$T(n) = O(n \log n)$$

✓ Let's prove that $T(n) \leq cn \log n$ for some $c > 0$.

❖ **Example)** MergeSort Time Complexity

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

✓ Guess

$$T(n) = O(n \lg n)$$

✓ Let's prove that $T(n) \le cn \lg n$ for some $c > 0$.

$$\begin{aligned} T(n) &= 2T\left(\frac{n}{2}\right) + n \\ &\le 2\left(c\frac{n}{2}\lg\frac{n}{2}\right) + n \\ &= cn\lg n - cn\lg 2 + n \\ &= cn\lg n - cn + n \\ &= cn\lg n - (c-1)n \\ &\le cn\lg n \quad \text{for} \quad c \ge 1 \end{aligned}$$

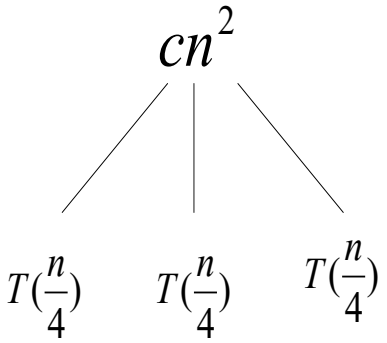
Recursion-tree Method

CONSTRUCTION OF A RECURSION TREE

❖ **Example)**

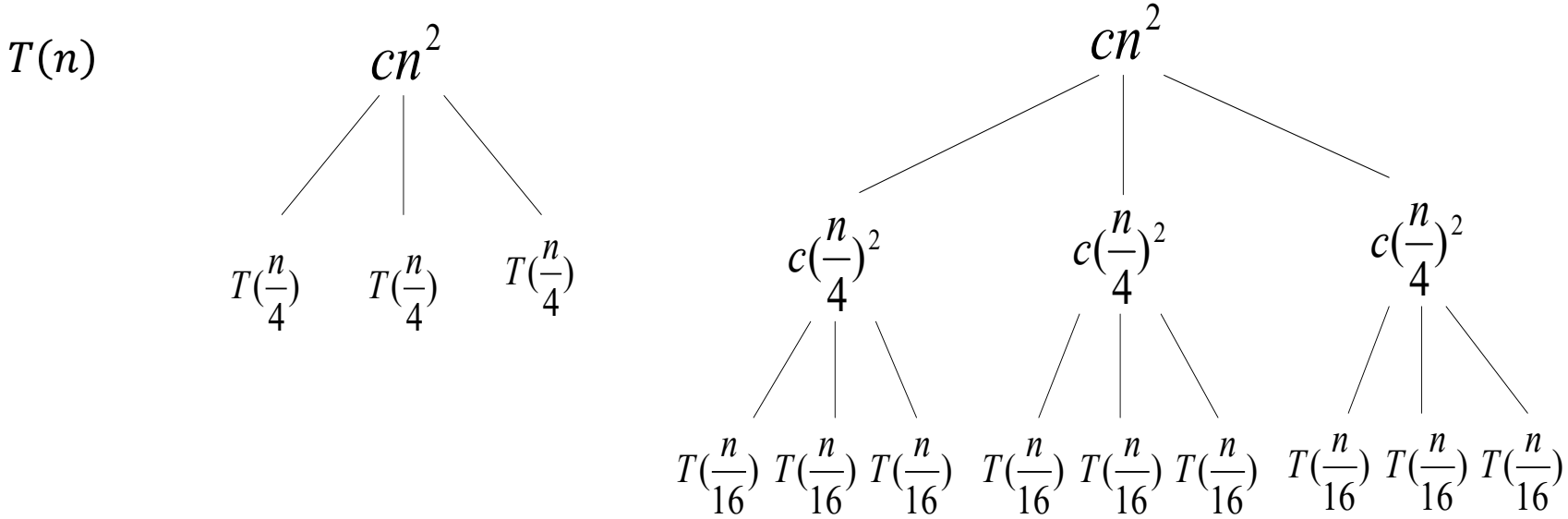
$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$$

$T(n)$



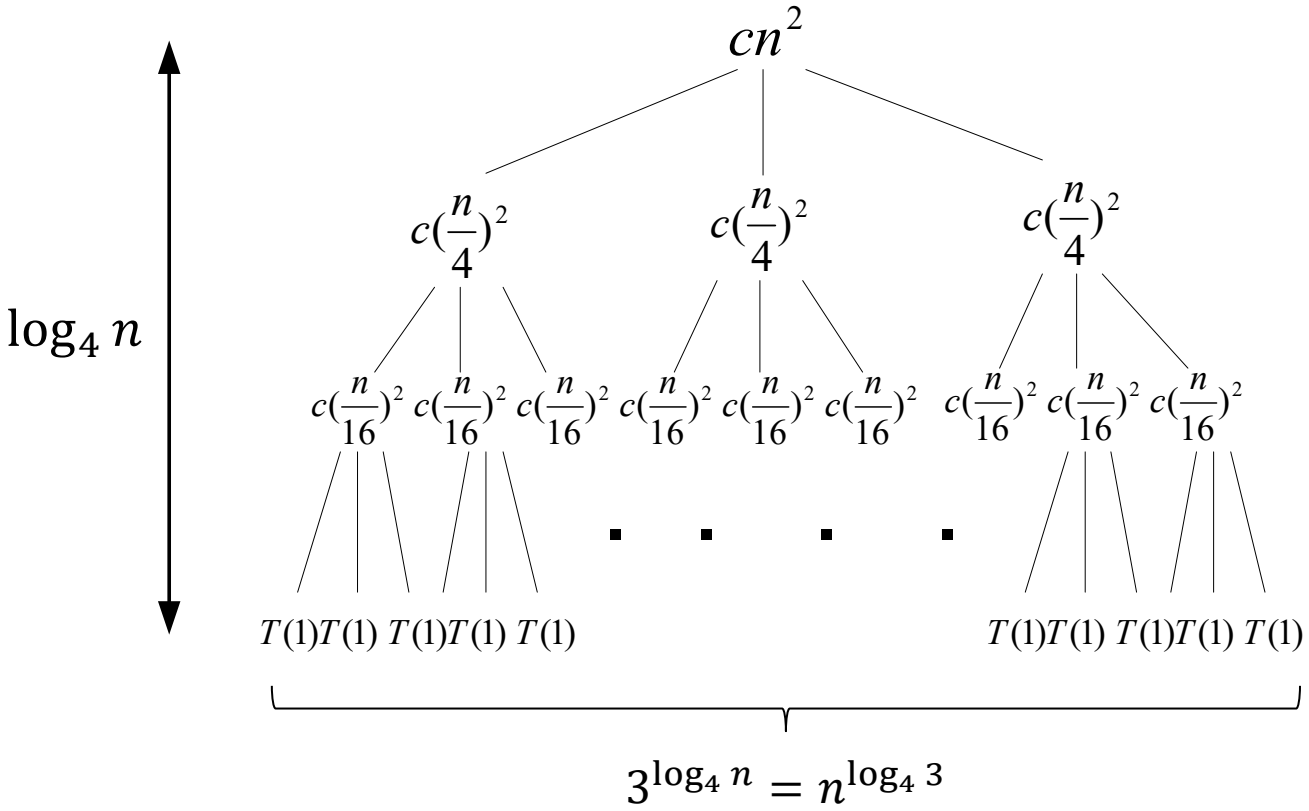
❖ **Example)**

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$$



❖ **Example)**

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$$



cn^2

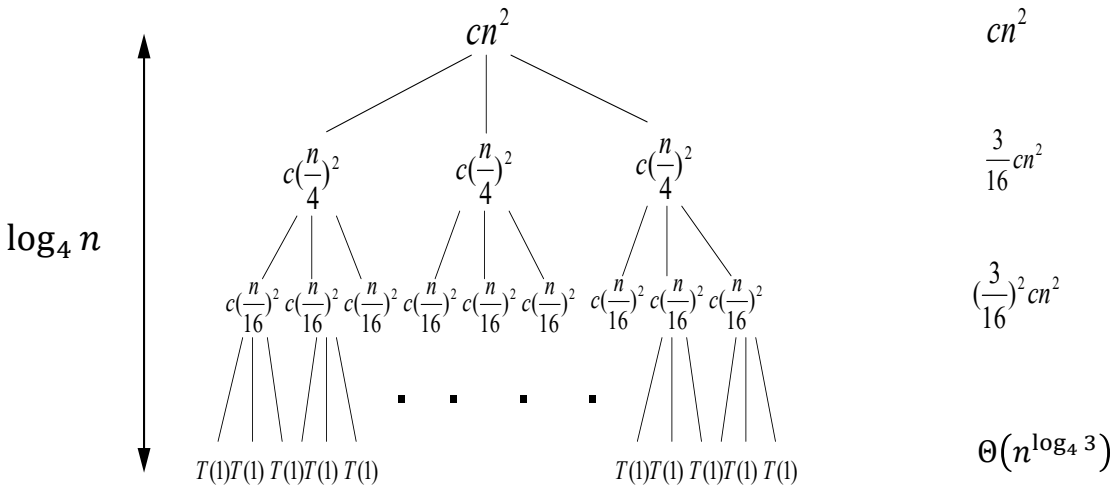
$\frac{3}{16}cn^2$

$(\frac{3}{16})^2 cn^2$

$\Theta(n^{\log_4 3})$

❖ **Example)**

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$$



$$= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3})$$

$T(n) = O(n^2)$ (read the section 4.2 in the textbook)

Can we verify this by using the substitution method?

RECURSION-TREE METHOD

- **Extra explanation link**
  - ✓ https://youtu.be/sLNPd_nPGIc

CONSTRUCTION OF A RECURSION TREE

- **Prove that**
  - ✓ $T(n) = O(n^2)$

    where $T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$
  - ✓ by using substitution method (The guess is also $T(n) = O(n^2)$).

Master Method

❖ Master theorem

- ✓ Let $a \geq 1$, $b > 1$ be constants, function $f(n) > 0$ and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$\boldsymbol{T(n) = aT(n/b) + f(n)}$$

- ✓ Then $T(n)$ can be bounded asymptotically as follows.

1. **If $f(n) = O\left(n^{(\log_b a)-\varepsilon}\right)$ for some constant $\varepsilon > 0$,**
   then $T(n) = \Theta\left(n^{\log_b a}\right)$.
2. **If $f(n) = \Theta\left(n^{\log_b a}\right)$**
   then $T(n) = \Theta\left(n^{\log_b a} \lg n\right)$.
3. **If $f(n) = \Omega\left(n^{(\log_b a)+\varepsilon}\right)$ for some constant $\varepsilon > 0$,** and if $f(n)$ satisfies the regularity condition $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n,
   then $T(n) = \Theta\left(f(n)\right)$

INTUITION

- **Compare $f(n)$ with $n^{log_b a}$**
- **Note that not all the cases for $f(n)$ are covered in the three cases.**

EXERCISE

- **Let's try some exercises**
  - ✓ $T(n) = 4T(n/2) + n$
  - ✓ $T(n) = 4T(n/2) + n^2$
  - ✓ $T(n) = 4T(n/2) + n^3$

- **Master theorem**
  - ✓ Let $a \geq 1$, $b > 1$ be constants, function $f(n) > 0$ and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n)$$

  - ✓ Then $T(n)$ can be bounded asymptotically as follows.

1. **If $f(n) = O(n^{(\log_b a)-\varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.**
2. **If $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \lg n)$.**
3. **If $f(n) = \Omega(n^{(\log_b a)+\varepsilon})$ for some constant $\varepsilon > 0$,** and if $f(n)$ satisfies the regularity condition $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, **then $T(n) = \Theta(f(n))$**

EXERCISE

- **What about…**
  - ✓ $T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$

Wrap-Up

WRAP-UP

- **Why do we study this chapter?**
  - ✓ We want to analyze algorithms that can be described as **recurrence** equations such as `MergeSort()`.

- **How?**
  - ✓ Iteration method
  - ✓ Substitution method
  - ✓ Recursion-tree method
  - ✓ Master method

- **Therefore,**
  - ✓ We can calculate asymptotic running (processing) time of those algorithms.

Thank you!